



Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатики и управления

КАФЕДРА ИУК3 Системы автоматического управления

ОТЧЕТ
по НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

НА ТЕМУ:

Исследование способностей рассуждения в малых языковых моделях

Студент группы ИУК3-72Б

(Подпись)

(И.О. Фамилия)

Руководитель НИР

(Подпись)

(И.О. Фамилия)

Оценка руководителя НИР

Баллов, 30-50

(Дата)

Оценка за защиту

Баллов, 30-50

(Дата)

Итоговая оценка

Баллов

(по пятибалльной шкале)

Комиссия:

(Подпись)

(И.О. Фамилия)

(Подпись)

(И.О. Фамилия)

(Подпись)

(И.О. Фамилия)

Калуга, 2025

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного
учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой ИУКЗ
()
« 05 » сентября 2025г.

ЗАДАНИЕ
на научно-исследовательскую работу
на тему:

Исследование способностей рассуждения в малых языковых моделях

Задание

- Разработать общую методику проведения исследования; выбрать последовательность и процедуры проведения исследования и обработки их результатов;
- Изучить основную информацию о методиках применения рассуждения, провести теоретические исследования; подобрать подходящую архитектуру модели; изучить основные техники внедрения рассуждений; собрать данные для обучения и тестирования модели; обработать полученные данные; обучить модель и получить метрики качества для сравнения с аналогами;
- Получить модель с рассуждающими способностями, провести сравнительный анализ полученной модели и аналогов, сопоставить результаты эксперимента с теоретическими исследованиями; сделать выводы;
- Подготовить доклад по материалам выполненных исследований; подготовить тезисы доклада и оформить их в соответствии с установленными требованиями; выступить с докладом на научной конференции.
- Выбрать периодическое рецензируемое издание, соответствующее выполненным исследованиям и индексируемое в российском индексе научного цитирования; подготовить материалы для научной публикации и оформить их в соответствии с установленными требованиями; пройти процедуры, связанные с перепиской с редакцией научного журнала, корректировкой рукописи и т.д.;
- Подготовить отчет и защитить результаты практики.

Дата выдачи задания « 5 » сентября 2025 г.

Руководитель 05.09.2025г.

(подпись, дата)

(И.О. Фамилия)

Студент 05.09.2025г.

(подпись, дата)

(И.О. Фамилия)

СОДЕРЖАНИЕ

Оглавление

ВВЕДЕНИЕ	4
1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ДИСТИЛЛЯЦИИ ЗНАНИЙ.....	7
1.1. Концепция дистилляции знаний	7
1.2. Методы рассуждений в языковых моделях	9
1.3. Метод низкоранговой адаптации (LoRA).....	10
2. МЕТОДОЛОГИЯ ИССЛЕДОВАНИЯ	13
2.1. Описание предлагаемого подхода.....	13
2.2. Создание обучающего датасета	14
2.3. Архитектура и параметры обучения	15
3. ЭКСПЕРИМЕНТАЛЬНЫЕ РЕЗУЛЬТАТЫ	16
3.1. Описание бенчмарков	16
3.2. Результаты экспериментов	16
3.3. Анализ результатов	17
ЗАКЛЮЧЕНИЕ.....	18
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	20
ПРИЛОЖЕНИЕ	22
Приложение А.....	22
Приложение Б	45

ВВЕДЕНИЕ

В последние годы наблюдается стремительное развитие и повсеместное внедрение систем обработки естественного языка (Natural Language Processing, NLP). Крупные языковые модели (Large Language Models, LLM), такие как GPT-4, Claude, Gemini демонстрируют впечатляющие способности в решении широкого спектра задач — от генерации текста до сложных математических вычислений и написания программного кода [1].

Однако широкое применение крупных языковых моделей сталкивается с рядом серьёзных ограничений. Основные проблемы связаны с колоссальными вычислительными затратами как на этапе их обучения, так и при последующем развёртывании, особенно на устройствах с ограниченными ресурсами. Современные модели могут содержать сотни миллиардов параметров, что делает их использование недоступным для большинства организаций и индивидуальных разработчиков.

Эффективным решением этих проблем является разработка и применение методов уменьшения размера нейронных сетей без значительной потери производительности. Один из наиболее перспективных подходов в этой области — дистилляция знаний (Knowledge Distillation) [2]. Суть этого метода заключается в том, что более компактная модель («ученик») обучается воспроизводить поведение значительно более крупной и мощной модели («учителя») [3, 4].

Актуальность данного исследования обусловлена необходимостью создания компактных, но при этом высокоэффективных языковых моделей, способных выполнять сложные

задачи рассуждений. Особый интерес представляет перенос не просто фактологических знаний, а именно способностей к рассуждению — навыков логического мышления, анализа и последовательного вывода заключений.

Целью настоящей работы является разработка и апробация нового подхода к дистилляции способностей к рассуждению от крупных языковых моделей к компактным, с использованием комбинации современных техник промптинга и инновационной стратегии самокоррекции.

Для достижения поставленной цели были сформулированы следующие задачи:

1. Провести анализ существующих методов дистилляции знаний и техник улучшения рассуждений языковых моделей.
2. Разработать методологию создания высококачественного обучающего датасета с использованием различных стратегий рассуждений.
3. Предложить и реализовать инновационную стратегию самокоррекции Razor-of-Thought (RoT).
4. Провести обучение компактной модели Qwen3-1.7B с использованием метода SFT и технологии LoRA.
5. Выполнить экспериментальную оценку эффективности предложенного подхода на стандартных бенчмарках.

Научная новизна работы заключается в предложении оригинальной стратегии самокоррекции Razor-of-Thought, которая обучает модель распознавать и исправлять типичные логические ошибки, а также в

комбинировании множества техник рассуждений (Chain-of-Thought, Program-of-Thought, Skip-Thinking) в едином обучающем датасете.

Практическая значимость работы состоит в создании эффективной модели, демонстрирующей производительность, сопоставимую с современными решениями в своём классе, при значительно меньших требованиях к вычислительным ресурсам.

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ДИСТИЛЛЯЦИИ ЗНАНИЙ

1.1. Концепция дистилляции знаний

Дистилляция знаний (Knowledge Distillation) — это метод сжатия модели машинного обучения, при котором компактная модель (называемая «учеником») обучается воспроизводить поведение более крупной предварительно обученной модели (называемой «учителем») или ансамбля моделей [2]. Данная концепция была формализована в работе Hinton et al. в 2015 году [4], хотя идеи сжатия знаний ансамбля моделей были предложены ещё в 2006 году. Схематичный вид дистилляции приведен на Рис. 1:

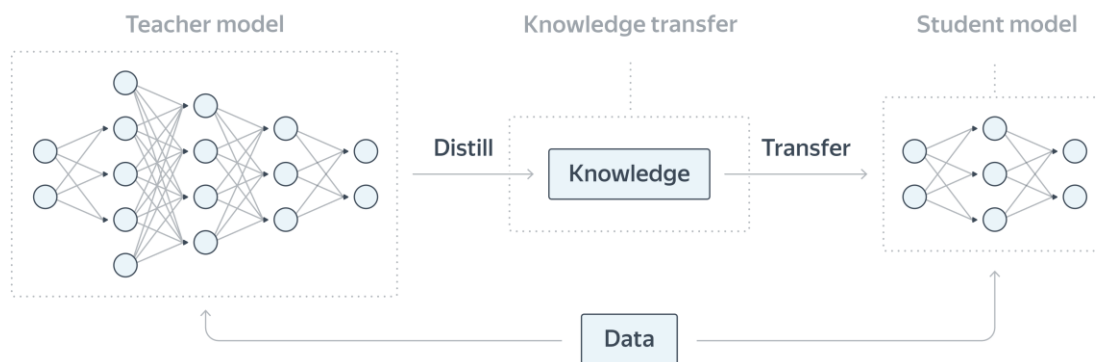


Рис. 1 – схематичное представление дистилляции из оригинальной статьи (https://yastatic.net/s3/education-portal/media/Artboard_30_f40ffa108f_4980246392.svg)

Основная идея дистилляции знаний заключается в том, что крупные модели обладают большей способностью к обучению компактных представлений знаний по сравнению с малыми моделями. При этом некоторая информация о компактном представлении знаний закодирована в псевдовероятностях, присваиваемых выходным переменным модели. Когда модель правильно предсказывает класс, она

присваивает большое значение выходной переменной, соответствующей этому классу, и меньшие значения другим выходным переменным [4].

Формально, для задачи классификации с использованием функции softmax с температурой, выходные вероятности модели определяются следующим образом:

$$y_i(x|t) = \exp(z_i(x)/t) / \sum_j \exp(z_j(x)/t)$$

где t — параметр температуры, $z_i(x)$ — логиты модели. При $t = 1$ получается стандартная функция softmax. При увеличении температуры распределение псевдовероятностей становится более «мягким», предоставляя больше информации о том, какие классы модель-учитель считает более похожими на предсказанный класс. Hinton называет это «тёмным знанием» (dark knowledge), которое передаётся модели-ученику в процессе дистилляции [4].

Существует три основных типа схем обучения при дистилляции знаний: офлайн-дистилляция, онлайн-дистилляция и самодистилляция. При офлайн-дистилляции (наиболее распространённый метод) предварительно обученная модель-учитель используется для обучения модели-ученика. При онлайн-дистилляции обе модели обучаются одновременно. При самодистилляции модель-учитель и модель-ученик представляют собой одну и ту же сеть [5].

В контексте данного исследования применяется офлайн-дистилляция, где в качестве моделей-учителей выступают крупные языковые модели с развитыми способностями к рассуждению, а модель-ученик обучается воспроизводить не только конечные ответы, но и сам процесс рассуждения.

1.2. Методы рассуждений в языковых моделях

Chain-of-Thought (CoT) промптинг — это техника, которая значительно улучшает способности языковых моделей к выполнению сложных задач, требующих многошаговых рассуждений [6]. Данный метод был предложен Wei et al. в 2022 году и основан на идее генерации цепочки промежуточных рассуждений перед получением финального ответа (см. Рисунок 2).



Рис. 2 – механизм работы механизма CoT

(<https://www.foundingminds.com/wp-content/uploads/2024/09/CoT-1.jpg>)

Ключевая концепция CoT заключается в том, что предоставление нескольких примеров (экземпляров), где процесс рассуждения явно показан, побуждает языковую модель включать шаги рассуждения в свои ответы. Исследования показали, что CoT промптинг может значительно повысить точность моделей на задачах арифметики, здравого смысла и символических рассуждений [6]. При этом выигрыш в производительности пропорционален размеру модели — модели с

параметрами около 100B и более демонстрируют наиболее значительные улучшения.

Program-of-Thought (PoT) — это расширение подхода CoT, которое делегирует вычислительные шаги внешнему интерпретатору языка программирования, такому как Python [7]. В отличие от CoT, где языковая модель выполняет как рассуждения, так и вычисления, PoT использует нулевой CoT для генерации и выполнения кода для получения финального ответа. Это особенно эффективно для задач, требующих точных математических вычислений или алгоритмических операций.

Skip-Thinking — это техника атомарного рассуждения, при которой мыслительный процесс разбивается на дискретные, независимые блоки [8]. Каждый блок заключается в специальные теги (например, `<chunk_start>` и `<chunk_end>`), что позволяет модели структурировать своё рассуждение более гибко и обеспечивает возможность параллельной обработки отдельных блоков мысли.

Self-Consistency (самосогласованность) — это подход, который задаёт модели один и тот же вопрос несколько раз и выбирает наиболее часто встречающийся результат в качестве финального ответа [9]. Агрегируя множество ответов на один и тот же запрос, можно получить более согласованный и надёжный ответ. Применение техники самосогласованности помогает уменьшить ошибки, вызванные неполным рассуждением.

1.3. Метод низкоранговой адаптации (LoRA)

Low-Rank Adaptation (LoRA) — это техника параметро-эффективной тонкой настройки, которая замораживает веса предварительно обученной модели и внедряет обучаемые матрицы

ранговой декомпозиции в слое архитектуры Transformer [10] (см. Рисунок 3). Данный метод был предложен исследователями Microsoft в 2021 году.

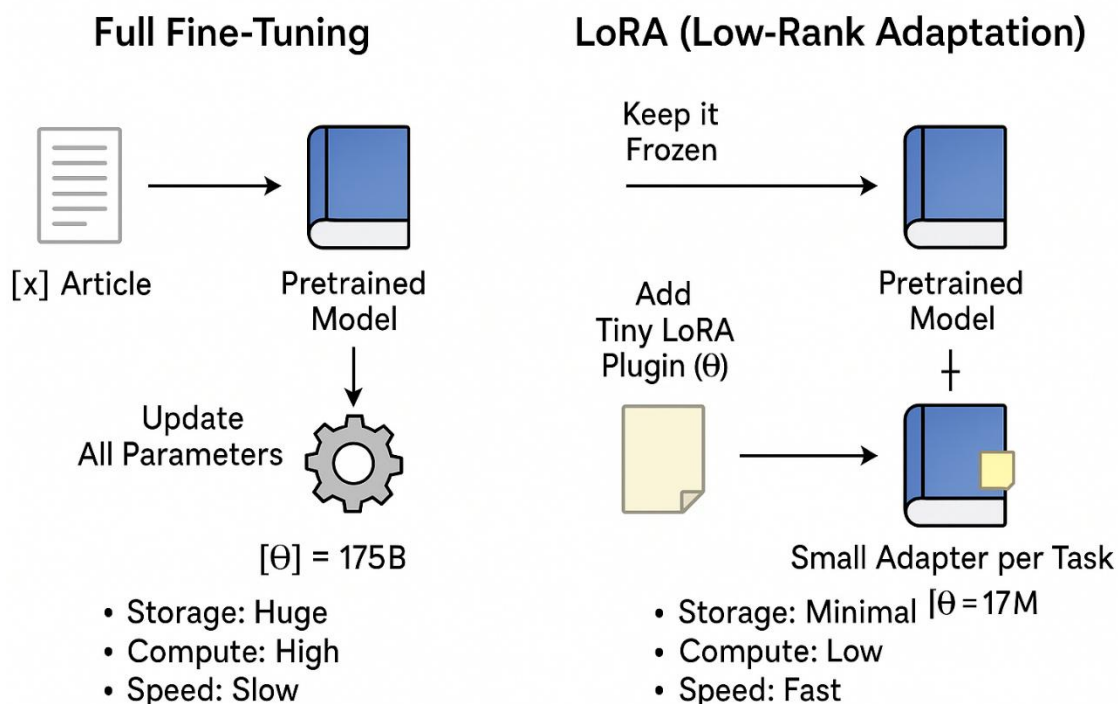


Рис. 3 – сравнение использования LoRA с полным переобучением весов сети (https://doimages.nyc3.cdn.digitaloceanspaces.com/010AI-ML/2025/Sharoni/17-jul/image_4.png)

Основная идея LoRA основана на понимании того, что крупные модели обладают низкоразмерной структурой. Используя меньшие матрицы (называемые низкоранговыми матрицами), LoRA эффективно адаптирует эти модели. Метод фокусируется на ключевой концепции: значительные изменения модели могут быть представлены меньшим количеством параметров, что делает процесс адаптации более эффективным [10].

Формально, для предварительно обученного веса $W_0 \in \mathbb{R}^{(d \times k)}$, LoRA ограничивает обновление путём представления его через низкоранговую декомпозицию:

$$W_0 + \Delta W = W_0 + BA, \text{ где } B \in \mathbb{R}^{(d \times r)}, A \in \mathbb{R}^{(r \times k)},$$

где $r \ll \min(d, k)$ — ранг. Во время обучения W_0 заморожен и не получает обновлений градиента, в то время как A и B содержат обучаемые параметры. На примере GPT-3 175B показано, что даже очень низкий ранг ($r = 1$ или 2) достаточен, даже когда полный ранг d достигает 12,288, что делает LoRA эффективным как по хранению, так и по вычислениям [10].

LoRA обладает рядом ключевых преимуществ:

- Сокращение количества обучаемых параметров до 10,000 раз по сравнению с полной тонкой настройкой.
- Снижение требований к GPU памяти примерно в 3 раза.
- Отсутствие дополнительной задержки при инференсе (в отличие от адаптеров).
- Возможность быстрого переключения между задачами путём замены матриц A и B .

2. МЕТОДОЛОГИЯ ИССЛЕДОВАНИЯ

2.1. Описание предлагаемого подхода

В настоящем исследовании предлагается комплексный подход к дистилляции способностей к рассуждению, который объединяет несколько современных техник промптинга с инновационной стратегией самокоррекции. Ключевым элементом предложенного метода является стратегия Razor-of-Thought (RoT) — оригинальная техника, направленная на обучение модели навыкам распознавания и исправления типичных логических ошибок.

Razor-of-Thought (RoT) — это предлагаемая стратегия, в рамках которой модель-учитель сначала намеренно генерирует ошибочное рассуждение, затем проводит его анализ, объясняет суть допущенной ошибки и предоставляет правильный ответ. Данная техника направлена на обучение модели-ученика:

- Распознавать типичные логические ошибки в рассуждениях.
- Понимать причины возникновения ошибок.
- Формировать навыки самокоррекции.
- Избегать распространённых логических ловушек.

Общая архитектура предложенного подхода включает следующие компоненты:

1. Генератор вопросов: использует модель-учитель для создания разнообразных задач из различных предметных областей.
2. Генератор рассуждений: применяет четыре стратегии (CoT, PoT, Skip-Thinking, RoT) для создания различных типов рассуждений.
3. Модуль самосогласованности: генерирует несколько вариантов ответа и выбирает наиболее консенсусный.

4. Модуль обучения: выполняет SFT с использованием LoRA для эффективной тонкой настройки.

2.2. Создание обучающего датасета

Основой исследования стал авторский датасет, сгенерированный с использованием пула из девяти API-ключей и многопоточной архитектуры для достижения максимальной эффективности. Датасет был создан для 35,000 уникальных вопросов из 10 различных предметных областей.

В качестве моделей-учителей выступили следующие передовые разработки:

- qwen-3-235b-a22b-thinking-2507 / gpt-oss-120b — для генерации пошаговых логических рассуждений.
- qwen-3-235b-a22b-instruct-2507 — для генерации разнообразных вопросов.
- qwen-3-coder-480b — для создания вопросов в области программирования и алгоритмов.

Предметные области датасета включают: логические загадки, простую математику, бытовые задачи на логику, задачи на пространственное мышление, физические парадоксы, задачи на оценку, программирование и алгоритмы, экономические сценарии, задачи на вероятность, простые математические вычисления.

В таблице 1 представлены форматы промптов для каждой стратегии рассуждения.

Таблица 1 — Форматы промптов для стратегий рассуждения

Стратегия	Описание формата
CoT	<thought_type>CoT</thought_type><think>Пошаговые рассуждения</think>Ответ
PoT	<thought_type>PoT</thought_type><think>Рассуждения с Python кодом</think>Ответ
Skip-Thinking	<thought_type>SkipThinking</thought_type><think><chunk_start>Блок</chunk_end></think>Ответ
RoT	<thought_type>RoT</thought_type><incorrect_think>Ошибка</incorrect_think><correction>Коррекция</correction>Ответ

2.3. Архитектура и параметры обучения

В качестве модели-ученика была выбрана Qwen3-1.7B, отличающаяся хорошим соотношением производительности и размера. Обучение проводилось с помощью техники Supervised Fine-Tuning (SFT) с использованием библиотеки Unsloth, реализующей метод низкоранговой адаптации (LoRA).

Параметры конфигурации LoRA представлены в таблице 2.

Таблица 2 — Параметры конфигурации LoRA

Параметр	Значение
Ранг (r)	16
lora_alpha	32
lora_dropout	0
Целевые модули	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Макс. длина последовательности	2048 токенов

Процесс обучения модели включал следующие параметры (таблица 3).

Таблица 3 — Параметры обучения модели

Параметр	Значение
Разделение датасета	98% train / 2% test
Оптимизатор	Lion 8bit
Скорость обучения	1e-4
Размер батча	8 (с gradient accumulation = 4)
Количество эпох	2-3
Функция потерь	Кросс-энтропийная

3. ЭКСПЕРИМЕНТАЛЬНЫЕ РЕЗУЛЬТАТЫ

3.1. Описание бенчмарков

Для оценки эффективности предложенного подхода использовались следующие бенчмарки:

- MathLogicQA — бенчмарк для оценки математических и логических рассуждений.
- ruMultiAr — многошаговые арифметические задачи на русском языке.
- ruModAr — задачи на модульную арифметику.
- MaMuRaMu — многоязычный математический бенчмарк.
- SimpleAr — простые арифметические вычисления.
- ruHumanEval — оценка способности генерации программного кода.

3.2. Результаты экспериментов

По результатам проведённых циклов обучения были получены показатели, представленные в таблице 4.

Таблица 4 — Сравнение качества дистилляции

Бенчмарк	Базовая	LoRA публ.	Adaptive	Adaptive+RoT
MathLogicQA	0.320	0.252	0.390	0.400
ruMultiAr	0.220	0.000	0.241	0.250
ruModAr	0.293	0.000	0.491	0.498
MaMuRaMu	0.344	0.278	0.542	0.580
SimpleAr	0.706	0.820	0.971	0.981
ruHumanEval	0.000	0.000	0.400	0.500

Результаты сравнения с крупными моделями при использовании интерпретатора Python представлены в таблице 5.

Таблица 5 — Сравнение с крупными моделями (с интерпретатором Python)

Модель	ruMultiAr	MathLogicQA
Qwen2.5-Max	0.591	0.775
GPT-4o	0.411	0.528
Qwen2.5-Math-1.5B	0.251	0.349
Qwen3-1.7B + LoRA + RoT + Python	0.620	0.520

3.3. Анализ результатов

Как видно из таблицы 4, модель, дообученная с использованием стратегий Adaptive Thinking и RoT, демонстрирует значительное превосходство над базовой моделью на всех представленных бенчмарках. На бенчмарке ruHumanEval прирост составил с 0.0 до 0.5 — модель научилась генерировать рабочий код на Python.

На бенчмарке MaMuRaMu прирост составил 68.6% относительно базовой модели (с 0.344 до 0.58), что свидетельствует о существенном улучшении многошаговых математических рассуждений. На бенчмарке ruModAr наблюдается улучшение на 70% (с 0.293 до 0.498).

Результаты таблицы 5 показывают, что предложенный подход с использованием интерпретатора Python позволяет малой модели (1.7B параметров) превосходить производительность таких крупных моделей, как GPT-4o, на специализированных задачах. На бенчмарке ruMultiAr наша модель (0.62) превосходит GPT-4o (0.411) на 50.9%.

ЗАКЛЮЧЕНИЕ

В ходе выполнения научно-исследовательской работы был успешно реализован и апробирован новый подход к дистилляции знаний, направленный на перенос сложных навыков рассуждения от крупных моделей-учителей к компактной модели-ученику.

Основные результаты работы:

1. Проведён анализ существующих методов дистилляции знаний и техник улучшения рассуждений языковых моделей.
2. Разработана методология создания высококачественного обучающего датасета, содержащего 35,000 уникальных вопросов из 10 предметных областей.
3. Предложена и реализована инновационная стратегия самокоррекции Razor-of-Thought (RoT).
4. Проведено обучение компактной модели Qwen3-1.7B с использованием метода SFT и технологии LoRA.
5. Выполнена экспериментальная оценка на стандартных бенчмарках, подтвердившая эффективность предложенного подхода.

Полученная модель демонстрирует производительность, сопоставимую с современными решениями в своём классе, при значительно меньших требованиях к вычислительным ресурсам. Модель превосходит базовую версию на 25-70% в зависимости от бенчмарка, а при использовании интерпретатора Python способна конкурировать с крупными моделями масштаба GPT-4o.

Направления дальнейших исследований включают: расширение набора стратегий рассуждений, применение методов обучения с

подкреплением (RLHF, DPO), а также исследование возможности переноса подхода на другие архитектуры.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Li, C., Chen, Q., Li, L., Wang, C., Li, Y., Chen, Z., Zhang, Y. Mixed Distillation Helps Smaller Language Model Better Reasoning // arXiv preprint arXiv:2312.10730, 2023.
2. Шиянов, В. Дистилляция знаний // Яндекс Практикум. — URL: <https://education.yandex.ru/handbook/ml/article/distillyaciya-znaniy>
3. Chen, X., Zhou, S., Liang, K., Sun, X., Liu, X. Skip-Thinking: Chunk-wise Chain-of-Thought Distillation // arXiv:2505.18642, 2025.
4. Hinton, G., Vinyals, O., Dean, J. Distilling the Knowledge in a Neural Network // arXiv:1503.02531, 2015.
5. Gou, J., Yu, B., Maybank, S.J., Tao, D. Knowledge Distillation: A Survey // IJCV, vol. 129, pp. 1789–1819, 2021.
6. Wei, J., Wang, X., Schuurmans, D., et al. Chain-of-Thought Prompting Elicits Reasoning in LLMs // NeurIPS 2022, arXiv:2201.11903.
7. Chen, W., Ma, X., Wang, X., Cohen, W.W. Program of Thoughts Prompting // arXiv:2211.12588, 2022.
8. Kojima, T., Gu, S.S., Reid, M., et al. Large Language Models are Zero-Shot Reasoners // NeurIPS 2022, arXiv:2205.11916.
9. Wang, X., Wei, J., et al. Self-Consistency Improves Chain of Thought Reasoning // ICLR 2023, arXiv:2203.11171.
10. Hu, E.J., Shen, Y., Wallis, P., et al. LoRA: Low-Rank Adaptation of Large Language Models // ICLR 2022, arXiv:2106.09685.
11. Vaswani, A., Shazeer, N., Parmar, N., et al. Attention Is All You Need // NeurIPS 2017, arXiv:1706.03762.
12. Devlin, J., Chang, M.W., Lee, K., Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers // NAACL 2019.

13. Brown, T.B., Mann, B., Ryder, N., et al. Language Models are Few-Shot Learners // NeurIPS 2020, arXiv:2005.14165.
14. Touvron, H., Lavril, T., Izacard, G., et al. LLaMA: Open and Efficient Foundation Language Models // arXiv:2302.13971, 2023.
15. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L. QLoRA: Efficient Finetuning of Quantized LLMs // NeurIPS 2023.

ПРИЛОЖЕНИЕ

Приложение А.

Исходный код генерации датасета

Полный исходный код генератора датасета.

```
# -*- coding: utf-8 -*-
```

```
import os
```

```
import json
```

```
import time
```

```
import random
```

```
import re
```

```
import threading
```

```
import traceback
```

```
import logging
```

```
from collections import deque
```

```
from concurrent.futures import ThreadPoolExecutor, as_completed
```

```
from tqdm import tqdm
```

```
# Попробуйте импортировать библиотеку, если она установлена
```

```
try:
```

```
    from cerebras.cloud.sdk import Cerebras
```

```
except ImportError:
```

```
    print("Библиотека Cerebras SDK не установлена. Пожалуйста, установите ее:")
```

```
    print("pip install cerebras-sdk")
```

```
    exit()
```

--- 🛠️ 1. ОСНОВНЫЕ НАСТРОЙКИ ---

#

=====
=====

API_KEYS = [

"csk-46pnxr3d25vrxeerxdwwd58w52rd4926wnw8cynkvpj2tm4m",

"csk-46djfx2mdvfpkvm89hfecw438phhnx98y9ptp53ne6hexk29",

"csk-4m292npy4p9ppxywyv354x9ve9vm628j8tn8yxdj2etvy69",

"csk-vmxprhkhc88vhf5mwe8y3n8khr5ctfjj9crmpbjr34645y6p",

"csk-6ycf68v2ntc3k6v6cy69e9er8f629rd2h224nm4hrhkkvp3x",

"csk-tcv2хypv8te8prcjchm2txpfrxyewc2ntcyek8ecrvc8dcxr",

"csk-trh3dw5dmhd889hwwvxkc2myv9ty6pr63f5hjxeyr6pn94mv",

"csk-5ey4vjf6je299jv5wed86jk82t22xnt8mcde8kwpd3vdnwh",

"csk-3nyjkxth3efwj2rr82m8r52jx32yhfm9ewce9jypfrhtctv9"

MODEL_NAME_THINKING = "gpt-oss-120b" # Для генерации ответов/рассуждений

MODEL_NAME_INSTRUCT = "qwen-3-235b-a22b-instruct-2507" # Для генерации
вопросов

MODEL_NAME_CODER = "qwen-3-coder-480b" # Специализированная модель
для программирования

--- 🚀 2. ПАРАМЕТРЫ ПРОИЗВОДИТЕЛЬНОСТИ ---

#

=====
=====

MAX_WORKERS = 15

MAX_RETRIES_PER_KEY = 3

RETRY_DELAY = 5

--- 📄 3. НАСТРОЙКИ ОГРАНИЧЕНИЯ СКОРОСТИ ---

#

=====

=====

REQUESTS_PER_SECOND = 2

REQUESTS_PER_MINUTE = 29

PERIOD_SECONDS_MINUTE = 60

REQUESTS_PER_HOUR = 900

PERIOD_SECONDS_HOUR = 3600

--- 📄 4. ПАРАМЕТРЫ ГЕНЕРАЦИИ ДАТАСЕТА ---

#

=====

=====

NUM_UNIQUE_QUESTIONS_TO_GENERATE = 20000

QUESTIONS_BATCH_SIZE = 5

NUM_REASONING_STRATEGIES_PER_QUESTION = 4

--- 🧠 5. НАСТРОЙКИ УЛУЧШЕНИЙ ---

#

=====

=====

NUM_SELF_CONSISTENCY_PATHS = 3

STRATEGY_PARAMS = {

 "CoT": {"temperature": 0.7},

 "PoT": {"temperature": 0.4},

 "SkipThinking": {"temperature": 0.6},


```
"RoT": {"temperature": 0.8}

}
```

ROT_GENERATION_WEIGHT = 1 # Во сколько раз чаще генерировать RoT по сравнению с каждой другой стратегией

--- 📁 6. ФАЙЛЫ ВЫВОДА И ЛОГИРОВАНИЯ ---

#

```
=====
=====
```

OUTPUT_DATASET_FILE = "reasoning_dataset_final.jsonl"

PROGRESS_FILE_QUESTIONS = "generated_questions_progress.json"

PROGRESS_FILE_REASONINGS = "generated_reasonings_progress.json"

EXTERNAL_DATASET_FILE = "qwen3_adaptive_reasoning_dataset.jsonl"

LOG_FILE = "dataset_generator.log" # Файл для сохранения логов

--- 🧠 7. ШАБЛОНЫ ПРОМПТОВ (УСИЛЕННЫЕ) ---

#

```
=====
=====
```

Общее требование для всех промптов: "Твой ответ ДОЛЖЕН быть ИСКЛЮЧИТЕЛЬНО валидным JSON."

Не добавляй никакого текста до или после JSON. Никаких markdown-блоков ```json ... ```"

QUESTION_GENERATOR_PROMPT = ""Твоя задача — придумать {count} уникальных и сложных вопросов на русском языке, требующих многошаговых рассуждений для ответа. Вопросы должны быть из следующей области: {topic}. Если тема 'простые математические вычисления', генерируй вопросы в виде сложных математических выражений. Сделай их разнообразными.

Твой ответ ДОЛЖЕН быть ИСКЛЮЧИТЕЛЬНО валидным JSON-массивом строк. Не добавляй никакого текста до или после JSON-массива."

REASONING_SOLVER_PROMPT_COT = ""Ты — экспертная система для пошаговых рассуждений (Chain-of-Thought). Реши задачу, подробно объяснив свой мыслительный процесс.

Вопрос для решения: {question}

Твой ответ ДОЛЖЕН быть ИСКЛЮЧИТЕЛЬНО валидным JSON-объектом следующего формата, без какого-либо дополнительного текста или markdown:

```
{{  
  "question": "{question}",  
  "chain_of_thought": "Твои пошаговые рассуждения здесь...",  
  "answer": "Только финальный, краткий ответ",  
  "thought_type": "CoT"  
}}
```

REASONING_SOLVER_PROMPT_POT = ""Ты — экспертная система для пошаговых рассуждений с кодом (Program-of-Thought). Реши задачу, используя блоки кода Python для вычислений.

Вопрос для решения: {question}

Твой ответ ДОЛЖЕН быть ИСКЛЮЧИТЕЛЬНО валидным JSON-объектом следующего формата, без какого-либо дополнительного текста или markdown:

```
{{  
  "question": "{question}",  
  "chain_of_thought": "Твои рассуждения с блоками ```python...``` здесь...",  
  "answer": "Только финальный, краткий ответ",  
  "thought_type": "PoT"  
}}
```

REASONING_SOLVER_PROMPT_SKIP_THINKING = ""Ты — экспертная система для блочных рассуждений (Skip-Thinking). Разбей свой мыслительный процесс на атомарные блоки, используя теги <chunk_start>, <chunk_end>, <continue_thinking> и <end_of_thought>.

Вопрос для решения: {question}

Твой ответ ДОЛЖЕН быть ИСКЛЮЧИТЕЛЬНО валидным JSON-объектом следующего формата, без какого-либо дополнительного текста или markdown:

```
{{  
  "question": "{question}",  
  "chain_of_thought": "  
1...</chunk_end><continue_thinking><chunk_start>Шар 2...</chunk_end><end_of_thought>",  
  "answer": "Только финальный, краткий ответ",  
  "thought_type": "SkipThinking"  
}}""
```

REASONING_SOLVER_PROMPT_ROT = ""Ты — экспертная система, которая учится на ошибках (Razor-of-Thought). Для заданной задачи: 1) сгенерируй НЕВЕРНОЕ рассуждение, 2) объясни, в чем ошибка, 3) дай правильный ответ.

Вопрос для решения: {question}

Твой ответ ДОЛЖЕН быть ИСКЛЮЧИТЕЛЬНО валидным JSON-объектом следующего формата, без какого-либо дополнительного текста или markdown:

```
{{  
  "question": "{question}",  
  "incorrect_chain_of_thought": "Здесь должно быть ошибочное рассуждение...",  
  "correction_explanation": "Здесь объяснение, почему рассуждение выше неверно...",  
  "correct_answer": "Только финальный ПРАВИЛЬНЫЙ ответ",  
  "thought_type": "RoT"  
}}""
```

```
REASONING_STRATEGIES = {

    "CoT": REASONING_SOLVER_PROMPT_COT,

    "PoT": REASONING_SOLVER_PROMPT_POT,

    "SkipThinking": REASONING_SOLVER_PROMPT_SKIP_THINKING,

    "RoT": REASONING_SOLVER_PROMPT_ROT,

}

QUESTION_TOPICS = [

    "логические загадки", "простая математика", "бытовые задачи на логику",

    "задачи на пространственное мышление", "физические парадоксы", "задачи на
оценку",

    "программирование и алгоритмы", # <-- Изменено для ясности

    "экономические сценарии", "задачи на вероятность",

    "простые математические вычисления"

]

CODER_TOPIC = "программирование и алгоритмы" # Константа для темы
программирования

# --- 🌐 ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ И УТИЛИТЫ ---

#
=====
====

API_KEY_MANAGERS = []

API_KEY_MANAGERS_LOCK = threading.Lock()

thread_local_client = threading.local()

# Настройка логирования

logging.basicConfig(
```

```
level=logging.INFO,  
  
format='%(asctime)s - %(levelname)s - %(message)s',  
  
handlers=[  
  
    logging.FileHandler(LOG_FILE, encoding='utf-8'),  
  
    logging.StreamHandler()  
  
]  
)
```

```
class RateLimiter:
```

```
    def __init__(self, limits: list[tuple[int, int]]):
```

```
        self.limits = limits
```

```
        self.timestamps_per_limit = [deque() for _ in limits]
```

```
    def get_wait_time(self) -> float:
```

```
        max_wait_time = 0.0
```

```
        now = time.monotonic()
```

```
        for i, (max_requests, period) in enumerate(self.limits):
```

```
            timestamps = self.timestamps_per_limit[i]
```

```
            while timestamps and timestamps[0] <= now - period:
```

```
                timestamps.popleft()
```

```
            if len(timestamps) >= max_requests:
```

```
                wait_time = timestamps[0] - (now - period) + (period / max_requests)
```

```
                if wait_time > max_wait_time:
```

```
                    max_wait_time = wait_time
```

```
        return max_wait_time
```

```
def record_request(self):  
  
    now = time.monotonic()  
  
    for timestamps in self.timestamps_per_limit:  
  
        timestamps.append(now)
```

```
class ApiKeyManager:  
  
    def __init__(self, api_key: str):  
  
        self.api_key = api_key  
  
        self.is_active = True  
  
        self.failure_count = 0  
  
        self.lock = threading.Lock()  
  
        self.rate_limiter = RateLimiter(limits=[  
  
            (REQUESTS_PER_SECOND, 1),  
  
            (REQUESTS_PER_MINUTE, PERIOD_SECONDS_MINUTE),  
  
            (REQUESTS_PER_HOUR, PERIOD_SECONDS_HOUR)  
  
        ])  
  
  
    def get_wait_time(self) -> float:  
  
        with self.lock:  
  
            return self.rate_limiter.get_wait_time()  
  
  
    def record_request(self):  
  
        with self.lock:  
  
            self.rate_limiter.record_request()  
  
  
    def record_failure(self):
```

```
with self.lock:

    self.failure_count += 1

    if self.failure_count >= MAX_RETRIES_PER_KEY:

        self.is_active = False

        logging.critical(f"Ключ {self.api_key[:8]}... отключен из-за многочисленных
сбоев.")
```

```
def record_success(self):
```

```
    with self.lock:
```

```
        self.failure_count = 0
```

```
def get_thread_local_client(api_key: str) -> Cerebras:
```

```
    if not hasattr(thread_local_client, 'client') or \
```

```
        not hasattr(thread_local_client, 'api_key') or \
```

```
        thread_local_client.api_key != api_key:
```

```
        thread_local_client.client = Cerebras(api_key=api_key)
```

```
        thread_local_client.api_key = api_key
```

```
    return thread_local_client.client
```

```
def load_progress(file_path):
```

```
    if os.path.exists(file_path):
```

```
        try:
```

```
            with open(file_path, 'r', encoding='utf-8') as f:
```

```
                return json.load(f)
```

```
        except (json.JSONDecodeError, IOError): return {}
```

```
    return {}
```

```
def save_progress(data, file_path):
```

```
    if isinstance(data, set):
```

```
        data = list(data)
```

```
    with open(file_path, 'w', encoding='utf-8') as f:
```

```
        json.dump(data, f, ensure_ascii=False, indent=2)
```

```
def safe_json_parse(response_content: str | None) -> dict | list | None:
```

```
    """
```

Более надежно извлекает и парсит JSON из строки с помощью регулярных выражений.

Обрабатывает случаи, когда JSON окружен текстом.

```
    """
```

```
    if not isinstance(response_content, str):
```

```
        return None
```

```
    try:
```

```
        # Ищем JSON объект или массив, игнорируя всё остальное
```

```
        match = re.search(r'(\{.*\})|(\[.*\])', response_content, re.DOTALL)
```

```
        if match:
```

```
            json_str = match.group(0)
```

```
            return json.loads(json_str)
```

```
        logging.warning(f"Не удалось найти JSON в ответе: {response_content[:200]}...")
```

```
        return None
```

```
    except (json.JSONDecodeError, IndexError) as e:
```

```
        logging.error(f"Ошибка парсинга JSON: {e}. Ответ: {response_content[:200]}...")
```



```
        return None

# --- CORE ЛОГИКА ---

#
=====
=====

def execute_api_call(prompt: str, task_desc: str, model_name: str, temperature: float,
max_tokens_output: int):

    selected_manager = None

    while True:

        min_wait_time = float('inf')

        best_manager_to_use_now = None

        best_manager_to_wait_for = None

    with API_KEY MANAGERS_LOCK:

        active_managers = [m for m in API_KEY MANAGERS if m.is_active]

        if not active_managers:

            logging.error("Все API ключи отключены или недоступны.")

            return None

        random.shuffle(active_managers)

        for manager in active_managers:

            wait_time = manager.get_wait_time()

            if wait_time == 0:

                best_manager_to_use_now = manager

                break

            if wait_time < min_wait_time:

                min_wait_time = wait_time
```

```
best_manager_to_wait_for = manager

if best_manager_to_use_now:

    selected_manager = best_manager_to_use_now

    selected_manager.record_request()

    break

if not best_manager_to_use_now and best_manager_to_wait_for:

    wait_duration = best_manager_to_wait_for.get_wait_time()

    time.sleep(wait_duration)

elif not best_manager_to_use_now and not best_manager_to_wait_for:

    # Этого не должно случиться, если есть активные ключи, но на всякий случай

    time.sleep(1)

try:

    client = get_thread_local_client(selected_manager.api_key)

    completion = client.chat.completions.create(

        messages=[{"role": "user", "content": prompt}],

        model=model_name,

        temperature=temperature,

        max_tokens=max_tokens_output,

    )

    selected_manager.record_success()

    return completion.choices[0].message.content.strip()

except Exception as e:
```

```
logging.warning(f"Ошибка с ключом {selected_manager.api_key[:8]}... ({task_desc}):  
{e}")  
  
selected_manager.record_failure()  
  
time.sleep(RETRY_DELAY)  
  
return None  
  
def generate_questions_batch(count: int, topic: str) -> list[str]:  
  
    prompt = QUESTION_GENERATOR_PROMPT.format(count=count, topic=topic)  
  
    # Выбираем модель: для кода - кодер, для остального - instruct  
  
    model_to_use = MODEL_NAME_CODER if topic == CODER_TOPIC else  
MODEL_NAME_INSTRUCT  
  
    logging.info(f"Генерация {count} вопросов по теме '{topic}' с моделью  
{model_to_use}.")  
  
    response = execute_api_call(prompt, f"Генерация вопросов", model_to_use,  
temperature=0.8, max_tokens_output=1024)  
  
    if not response: return []  
  
    questions = safe_json_parse(response)  
  
    return [q.strip() for q in questions if isinstance(q, str)] if isinstance(questions, list) else []  
  
def generate_reasoning_path(question: str, strategy_type: str, prompt_template: str, topic:  
str, cache: dict) -> dict | None:  
  
    # Выбираем модель: для кода - кодер, для остального - thinking  
  
    model_to_use = MODEL_NAME_CODER if topic == CODER_TOPIC else  
MODEL_NAME_THINKING
```

```
prompt = prompt_template.format(question=question)

params = STRATEGY_PARAMS.get(strategy_type, {"temperature": 0.7})

    response = execute_api_call(prompt, f"Генерация {strategy_type}", model_to_use,
**params, max_tokens_output=2048)

    if not response: return None

    data = safe_json_parse(response)

    if not isinstance(data, dict):

        logging.warning(f"Не удалось распарсить JSON для {strategy_type} по вопросу:
{question[:50]}...")

        return None

    return process_reasoning_result(data, strategy_type, question, cache)

def process_reasoning_result(data: dict, strategy_type: str, question: str, cache: dict) -> dict
| None:

    cache_key = f"{question}|{strategy_type}"

    if strategy_type == "RoT":

        keys = ["incorrect_chain_of_thought", "correction_explanation", "correct_answer"]

        if not all(k in data for k in keys): return None

        result = {

            "chain_of_thought": data["incorrect_chain_of_thought"],

            "correction": data["correction_explanation"],

            "answer": data["correct_answer"],

            "thought_type": "RoT"
```

```
    }

else:

    keys = ["chain_of_thought", "answer"]

    if not all(k in data for k in keys): return None

    result = {

        "chain_of_thought": data['chain_of_thought'],

        "answer": data['answer'],

        "thought_type": data.get('thought_type', strategy_type)

    }

if result:

    cache[cache_key] = result

    return result

return None


def generate_self_consistent_example(question: str, strategy_type: str, prompt_template:
str, topic: str, cache: dict) -> dict | None:

    paths = []

    with ThreadPoolExecutor(max_workers=NUM_SELF_CONSISTENCY_PATHS) as
executor:

        future_to_task = {executor.submit(generate_reasoning_path, question, strategy_type,
prompt_template, topic, cache): i for i in range(NUM_SELF_CONSISTENCY_PATHS)}

    for future in as_completed(future_to_task):

        try:

            result = future.result()

            if result: paths.append(result)
```

```
except Exception:

    logging.warning(f"Сбой одного из путей в self-consistency для вопроса:
'{question[:50]}...'")

    logging.debug(traceback.format_exc())

if not paths: return None

answer_counts = {}

for path in paths:

    answer_str = str(path.get('answer', ""))

    answer_counts[answer_str] = answer_counts.get(answer_str, 0) + 1

most_common_answer = max(answer_counts, key=answer_counts.get)

# Если консенсуса нет (все ответы разные), отбрасываем пример
if answer_counts[most_common_answer] < 2 and len(paths) > 1:

    logging.info(f"Нет консенсуса в self-consistency для вопроса '{question[:50]}...'.
Пример отброшен.")

    return None

chosen_path = next((p for p in paths if str(p.get('answer', "")) == most_common_answer),
None)

if not chosen_path: return None

if chosen_path['thought_type'] == 'RoT':

    content = (

        f"<thought_type>RoT</thought_type>\n"

        f"<incorrect_think>\n{chosen_path['chain_of_thought']}\n</incorrect_think>\n"
```

```

        f"<correction>\n{chosen_path['correction']}\n</correction>\n"

        f"{chosen_path['answer']}"

    )

else:

    content = (

        f"<thought_type>{chosen_path['thought_type']}</thought_type>\n"

        f"<think>\n{chosen_path['chain_of_thought']}\n</think>\n"

        f"{chosen_path['answer']}"

    )

    return {"messages": [{"role": "user", "content": question}, {"role": "assistant", "content":
content}]}

```

--- 🚀 ОСНОВНОЙ СКРИПТ ---

#

```

=====
=====

```

```

def main():

    global API_KEY MANAGERS

    logging.info("=*80)

    logging.info(" 🚀 Запуск генератора датасета для обучения рассуждениям")

    logging.info(f" Лимиты на ключ: {REQUESTS_PER_SECOND}/сек,
{REQUESTS_PER_MINUTE}/мин, {REQUESTS_PER_HOUR}/час.")

    logging.info("=*80)

    if not API_KEYS or "csk-" not in API_KEYS[0]:

```

```
logging.critical("API ключи Cerebras не заданы в списке API_KEYS. Выход.")

return

API_KEY MANAGERS = [ApiKeyManager(key) for key in API_KEYS]

logging.info(f"🔑 Загружено {len(API_KEY MANAGERS)} ключей в пул.")

logging.info(f"\n[Этап 1/4] Генерация {NUM_UNIQUE_QUESTIONS_TO_GENERATE}
уникальных вопросов...")

all_questions_set =
set(load_progress(PROGRESS_FILE_QUESTIONS).get('questions', []))

logging.info(f"📄 Найдено {len(all_questions_set)} вопросов в файле прогресса.")

with tqdm(total=NUM_UNIQUE_QUESTIONS_TO_GENERATE, desc="Генерация
вопросов") as pbar:

    pbar.update(len(all_questions_set))

    while len(all_questions_set) < NUM_UNIQUE_QUESTIONS_TO_GENERATE:

        needed = NUM_UNIQUE_QUESTIONS_TO_GENERATE - len(all_questions_set)

        batch_size = min(needed, QUESTIONS_BATCH_SIZE)

        topic = random.choice(QUESTION_TOPICS)

        new_batch = generate_questions_batch(batch_size, topic)

        if all(not m.is_active for m in API_KEY MANAGERS):

            logging.error("Все API ключи были отключены. Генерация вопросов
остановлена.")

            break

        if not new_batch:
```



```
        continue

    prev_count = len(all_questions_set)

    all_questions_set.update(new_batch)

    if len(all_questions_set) > prev_count:

        pbar.update(len(all_questions_set) - prev_count)

        save_progress({'questions':    list(all_questions_set)},
PROGRESS_FILE_QUESTIONS)

    questions_list = list(all_questions_set)

    logging.info(f"✅ [Этап 1] Завершен. Сгенерировано {len(questions_list)} вопросов.")

    if not questions_list:

        logging.warning("Вопросы не были сгенерированы. Завершение работы.")

        return

    logging.info(f"\n[Этап 2/4] Подготовка задач на генерацию рассуждений...")

    tasks = []

    # --- ИЗМЕНЕННАЯ ЛОГИКА: ВЗВЕШЕННЫЙ ВЫБОР СТРАТЕГИЙ ---

    all_strategies = list(REASONING_STRATEGIES.keys())

    other_strategies = [s for s in all_strategies if s != "RoT"]

    # Создаем взвешенный пул, где RoT встречается чаще

    weighted_strategy_pool = ["RoT"] * ROT_GENERATION_WEIGHT + other_strategies
```

```
logging.info(f"Пул стратегий для выбора: {weighted_strategy_pool}. RoT будет  
генерироваться чаще.")
```

```
for q in questions_list:
```

```
    # Определяем тему вопроса (упрощенная логика)
```

```
    question_topic = CODER_TOPIC if any(kw in q.lower() for kw in ['код', 'программ',  
'алгоритм', 'python', 'данных']) else 'general'
```

```
    # Выбираем K стратегий из взвешенного пула (с возможностью повторений)
```

```
        chosen_strategies = random.choices(weighted_strategy_pool,  
k=NUM_REASONING_STRATEGIES_PER_QUESTION)
```

```
for strategy in chosen_strategies:
```

```
    tasks.append({
```

```
        "question": q,
```

```
        "strategy_type": strategy,
```

```
        "prompt_template": REASONING_STRATEGIES[strategy],
```

```
        "topic": question_topic
```

```
    })
```

```
random.shuffle(tasks)
```

```
logging.info(f"💡 [Этап 2] Завершен. Сформировано {len(tasks)} задач.")
```

```
logging.info(f"\n[Этап 3/4] Запуск генерации рассуждений в {MAX_WORKERS}  
поточков...")
```

```
final_samples = []
```

```
reasoning_cache = load_progress(PROGRESS_FILE_REASONINGS)
```

```
with ThreadPoolExecutor(max_workers=MAX_WORKERS) as executor:

    future_to_task = {executor.submit(generate_self_consistent_example, t['question'],
t['strategy_type'], t['prompt_template'], t['topic'], reasoning_cache): t for t in tasks}

    for future in tqdm(as_completed(future_to_task), total=len(tasks), desc="Генерация
рассуждений"):

        try:

            result = future.result()

            if result: final_samples.append(result)

        except Exception as e:

            logging.error(f"Критическая ошибка в потоке: {e}")

            logging.debug(traceback.format_exc())

    logging.info(f"✅ [Этап 3] Завершен. Сгенерировано {len(final_samples)} валидных
примеров.")

    logging.info(f"\n[Этап 4/4] Объединение с внешним датасетом и запись
результатов...")

    if os.path.exists(EXTERNAL_DATASET_FILE):

        try:

            with open(EXTERNAL_DATASET_FILE, 'r', encoding='utf-8') as f:

                external_samples = [json.loads(line) for line in f]

                logging.info(f"📂 Загружено {len(external_samples)} примеров из внешнего
файла.")

            final_samples.extend(external_samples)

        except (json.JSONDecodeError, IOError) as e:
```

```
logging.error(f"Не удалось загрузить внешний датасет: {e}")

else:

    logging.info(f"Внешний файл '{EXTERNAL_DATASET_FILE}' не найден, будет
записан только сгенерированный датасет.")

with open(OUTPUT_DATASET_FILE, 'a', encoding='utf-8') as f:

    for entry in final_samples:

        f.write(json.dumps(entry, ensure_ascii=False) + '\n')

save_progress(reasoning_cache, PROGRESS_FILE_REASONINGS)

        logging.info(f"        Кэш        рассуждений        сохранен        в        файл:
{PROGRESS_FILE_REASONINGS}")

logging.info("\n" + "="*80)

logging.info("🎉 Генерация и объединение завершено!")

logging.info(f"💾 Итоговый датасет сохранен в файл: {OUTPUT_DATASET_FILE}")

if final_samples:

    logging.info("\n 📄 Пример структуры одной записи:")

    logging.info(json.dumps(final_samples[0], indent=2, ensure_ascii=False))

logging.info("="*80)

if __name__ == "__main__":

    main()
```

Приложение Б

Исходный код обучения модели

Полный исходный код обучения модели

```
# -*- coding: utf-8 -*-
```

```
"""Qwen3_SFT_Part1.ipynb
```

Automatically generated by Colab.

Original file is located at

https://colab.research.google.com/drive/1fYIWHWc6q4RD_7_SDcpb6WzThe8MIOuq

```
"""
```

```
!pip install unsloth trl transformers datasets
```

```
# =====
```

```
# Шаг 0: Настройка и импорты
```

```
# =====
```

```
import os
```

```
import torch
```

```
from google.colab import drive
```

```
from unsloth import FastLanguageModel
```

```
from trl import SFTTrainer
```

```
from transformers import TrainingArguments
```

```
from datasets import load_dataset
```

```
# Подключаем Google Drive

drive.mount('/content/drive')


# Определяем базовые пути к проекту для удобства

PROJECT_PATH = "/content/drive/MyDrive/Qwen3_Reasoning_Project/"

DATA_PATH = PROJECT_PATH + "data/"

SFT_OUTPUT_PATH = PROJECT_PATH + "sft_outputs/"

FINAL_MODEL_PATH = PROJECT_PATH + "sft_model/qwen3_1.7b_reasoning_sft_final"


# Убедимся, что директории существуют

os.makedirs(DATA_PATH, exist_ok=True)

os.makedirs(SFT_OUTPUT_PATH, exist_ok=True)

os.makedirs(os.path.dirname(FINAL_MODEL_PATH), exist_ok=True)


# =====

# Шаг 1: Подготовка, перемешивание и разделение датасета

# =====

print(">>> Шаг 1: Подготовка датасета <<<")


# Пути к файлам

source_dataset_file = DATA_PATH + "reasoning_dataset_final.jsonl"

train_file = DATA_PATH + "train.jsonl"

test_file = DATA_PATH + "test.jsonl"


# Параметры разделения

TEST_SIZE = 0.02 # 2% данных для теста
```

```
RANDOM_SEED = 42 # Для воспроизводимости результатов

# Проверяем, существуют ли уже разделенные файлы

if os.path.exists(train_file) and os.path.exists(test_file):

    print(f"✅ Файлы '{os.path.basename(train_file)}' и '{os.path.basename(test_file)}' уже  
существуют. Пропускаем шаг разделения.")

else:

    print(f"Файлы для обучения/теста не найдены. Начинаем процесс разделения...")

    if not os.path.exists(source_dataset_file):

        raise FileNotFoundError(f"Исходный файл датасета не найден по пути:  
{source_dataset_file}. Пожалуйста, загрузите его.")

# 1. Загружаем исходный датасет

print(f"- Загрузка исходного файла: {os.path.basename(source_dataset_file)}")

full_dataset = load_dataset("json", data_files=source_dataset_file, split="train")

# 2. Перемешиваем датасет

print(f"- Перемешивание датасета (seed={RANDOM_SEED})...")

shuffled_dataset = full_dataset.shuffle(seed=RANDOM_SEED)

# 3. Разделяем на обучающую и тестовую выборки

print(f"- Разделение на train/test (test_size={TEST_SIZE})...")

split_dataset = shuffled_dataset.train_test_split(test_size=TEST_SIZE)

# 4. Сохраняем на Google Drive

print(f"- Сохранение '{os.path.basename(train_file)}'...")
```

```
split_dataset["train"].to_json(train_file, orient="records", lines=True)

print(f"- Сохранение '{os.path.basename(test_file)}'...")

split_dataset["test"].to_json(test_file, orient="records", lines=True)

print("✅ Разделение и сохранение завершено!")

# =====

# Шаг 2: Загрузка модели и токенизатора

# =====

print("\n>>> Шаг 2: Загрузка модели и токенизатора Unsloth <<<")

model, tokenizer = FastLanguageModel.from_pretrained(

    model_name="Qwen/Qwen3-1.7B",

    max_seq_length=2048,

    dtype=None,

    load_in_4bit=True,

)

print("✅ Модель и токенизатор загружены.")

# =====

# Шаг 3: Конфигурация LoRA

# =====

print("\n>>> Шаг 3: Конфигурация PEFT (LoRA) <<<")

model = FastLanguageModel.get_peft_model(

    model,
```



```
r=16,

target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],

lora_alpha=32,

lora_dropout=0,

bias="none",

use_gradient_checkpointing=True,

)

print("✅ LoRA-адаптеры добавлены к модели.")


# =====

# Шаг 4: Загрузка подготовленного датасета

# =====

print("\n>>> Шаг 4: Предварительное форматирование датасета <<<")


# Функция теперь принимает `examples` (множественное число)

def formatting_prompts_func(examples):

    conversations = examples["messages"]

    # Создаем список отформатированных строк для всего батча

    texts = [tokenizer.apply_chat_template(conv, tokenize=False, add_generation_prompt=False)
for conv in conversations]

    # Возвращаем словарь с новым ключом 'text'

    return {"text": texts}


# Загружаем датасет

dataset = load_dataset("json", data_files={"train": train_file, "test": test_file})
```

```
# Применяем форматирование ко всему датасету ОДИН РАЗ

dataset = dataset.map(formatting_prompts_func, batched=True,)

print("✅ Датасет отформатирован.")

print(dataset)

print("Пример отформатированного текста:", dataset["train"][0]["text"])


# =====

# Шаг 5: Настройка и запуск SFTTrainer (ИЗМЕНЕННЫЙ БЛОК)

# =====

print("\n>>> Шаг 5: Запуск процесса SFT <<<")

trainer = SFTTrainer(

    model=model,

    tokenizer=tokenizer,

    train_dataset=dataset["train"],

    eval_dataset=dataset["test"],

    dataset_text_field="text",

    max_seq_length=2048,

    args=TrainingArguments(

        per_device_train_batch_size=8,

        gradient_accumulation_steps=4,

        warmup_steps=10,

        num_train_epochs=2,

        # =====

        # ИЗМЕНЕНИЯ ЗДЕСЬ

        # =====

        # Для Lion часто рекомендуют lr поменьше и weight_decay побольше
```

```
learning_rate = 1e-4,    # <--- Уменьшили lr (например, со 2e-4 до 1e-4)

weight_decay = 0.01,    # <--- Добавили/убедились, что есть weight_decay

optim = "lion_8bit",    # <--- ВАШ НОВЫЙ ОПТИМИЗАТОР

# =====

fp16=not torch.cuda.is_bf16_supported(),

bf16=torch.cuda.is_bf16_supported(),

logging_steps=10,

eval_strategy="steps",

eval_steps=50,

output_dir=SFT_OUTPUT_PATH,

save_strategy="steps",

save_steps=50,

report_to="none"

),

)

# Запускаем обучение

trainer.train()

# =====

# Шаг 6: Сохранение финальной модели на Google Drive

# =====

print("\n>>> Шаг 6: Сохранение финальной модели <<<")

model.save_pretrained(FINAL_MODEL_PATH)

tokenizer.save_pretrained(FINAL_MODEL_PATH)
```

```
print("🎉" * 3)

print(f"Обучение SFT успешно завершено! Модель сохранена в: {FINAL_MODEL_PATH}")

print("🎉" * 3)


import time # <--- Добавляем импорт time для задержки

import traceback # <--- Добавляем для логирования ошибок

from google.colab import runtime


print("="*80)

print("✅ Скрипт завершил свою работу (успешно или с ошибкой).")

print("👉 Отключение от среды выполнения Google Colab через 5 секунд...")

print("="*80)


time.sleep(5)


# Команда полного отключения

runtime.unassign()


"""Проверка Qwen_SFT"""


# -*- coding: utf-8 -*-


"""

Сравнение SFT-Qwen3 и базовой Qwen3-1.7B

"""
```

```
import torch

from transformers import AutoTokenizer, AutoModelForCausalLM


# Пути к моделям

SFT_MODEL_PATH =
"/content/drive/MyDrive/Qwen3_Reasoning_Project/sft_model/qwen3_1.7b_reasoning_sft_final"

BASE_MODEL_NAME = "Qwen/Qwen3-1.7B"


# Настройки

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

DTYPE = torch.bfloat16 if torch.cuda.is_available() else torch.float32


def load_model(model_path, is_local=True):

    print(f"Загружаю модель: {model_path}")

    tokenizer = AutoTokenizer.from_pretrained(model_path)

    model = AutoModelForCausalLM.from_pretrained(

        model_path,

        torch_dtype=DTYPE,

        device_map="auto" if torch.cuda.is_available() else None,

    )

    return tokenizer, model


# Загружаем обе модели

sft_tokenizer, sft_model = load_model(SFT_MODEL_PATH)

base_tokenizer, base_model = load_model(BASE_MODEL_NAME, is_local=False)
```

Тестовые задачи

test_prompts = [

"Если спутник движется на высоте 36000 км над планетой с радиусом 6052 км и массой, равной 0.82 массы Земли, как изменится ускорение свободного падения на этой высоте по сравнению с поверхностью планеты?",

"Реши выражение: $((7 \times (3 + 2))^2 - (4^3 \div 2)) \div (5 + (6 \times (8 - 5))) + \sqrt{(169) - |-11|}$ ",

"Опиши пошагово алгоритм быстрой сортировки массива (сортировка Хоара).",

]

def generate_answer(tokenizer, model, prompt, max_new_tokens=2048):

inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

with torch.no_grad():

outputs = model.generate(

 **inputs,

 max_new_tokens=max_new_tokens,

 temperature=0.7,

 top_p=0.9,

 do_sample=True,

 eos_token_id=tokenizer.eos_token_id,

)

return tokenizer.decode(outputs[0], skip_special_tokens=True)

if __name__ == "__main__":

for i, q in enumerate(test_prompts, 1):

 print("=" * 100)

 print(f"[Q{i}] {q}\n")

```
# Базовая модель

base_ans = generate_answer(base_tokenizer, base_model, q)

print(f"[BASE] {base_ans}\n")


# SFT-модель

sft_ans = generate_answer(sft_tokenizer, sft_model, q)

print(f"[SFT] {sft_ans}\n")


print(tokenizer.eos_token, tokenizer.eos_token_id)
```